



Production Information Systems

Topic : Object-oriented analysis and design

Chapter 7 : Unified Modeling Language

ISE Department
Year 4/ Level 7

Instructor : Dr. Jomana Bashatah



Outline

1. Introduction
2. Sequence Diagram
3. Activity Diagram
4. Case Study

1.Introduction

- As discussed in last lecture, we will focus through this topic on a subset of UML diagrams that are useful in information system design.
- The relevant subset includes use case diagrams, sequence diagrams, activity diagrams, state chart diagrams, and class diagrams.
- The Use case diagram, which represents a description of how users interact with the system, considers actors who use the system and the functions that the actor calls upon the system to perform.

2.Sequence Diagram

- A use case diagram is a broad description of the functions to be performed by the system and the external users (actors) of those functions.
- To obtain a more refined understanding of the various possible interactions that may occur between the actors and the system during the execution of a use case, it is necessary to construct **scenarios**.
- A **scenario** is a description of a sequence of actions that illustrates the execution of a use case instance. In most use cases, several possible scenarios may occur.

2.Sequence Diagram

- For example, in trying to make a ticket sale, it is possible that the sale will be made without complication or that some complication will occur, for example, the submitted credit card information could be invalid.
- The normal use case, in which all goes well, is one scenario; the case of the rejected credit card is another scenario.
- Next Figure illustrates the fact that out of one use case there may be one or more possible scenarios.

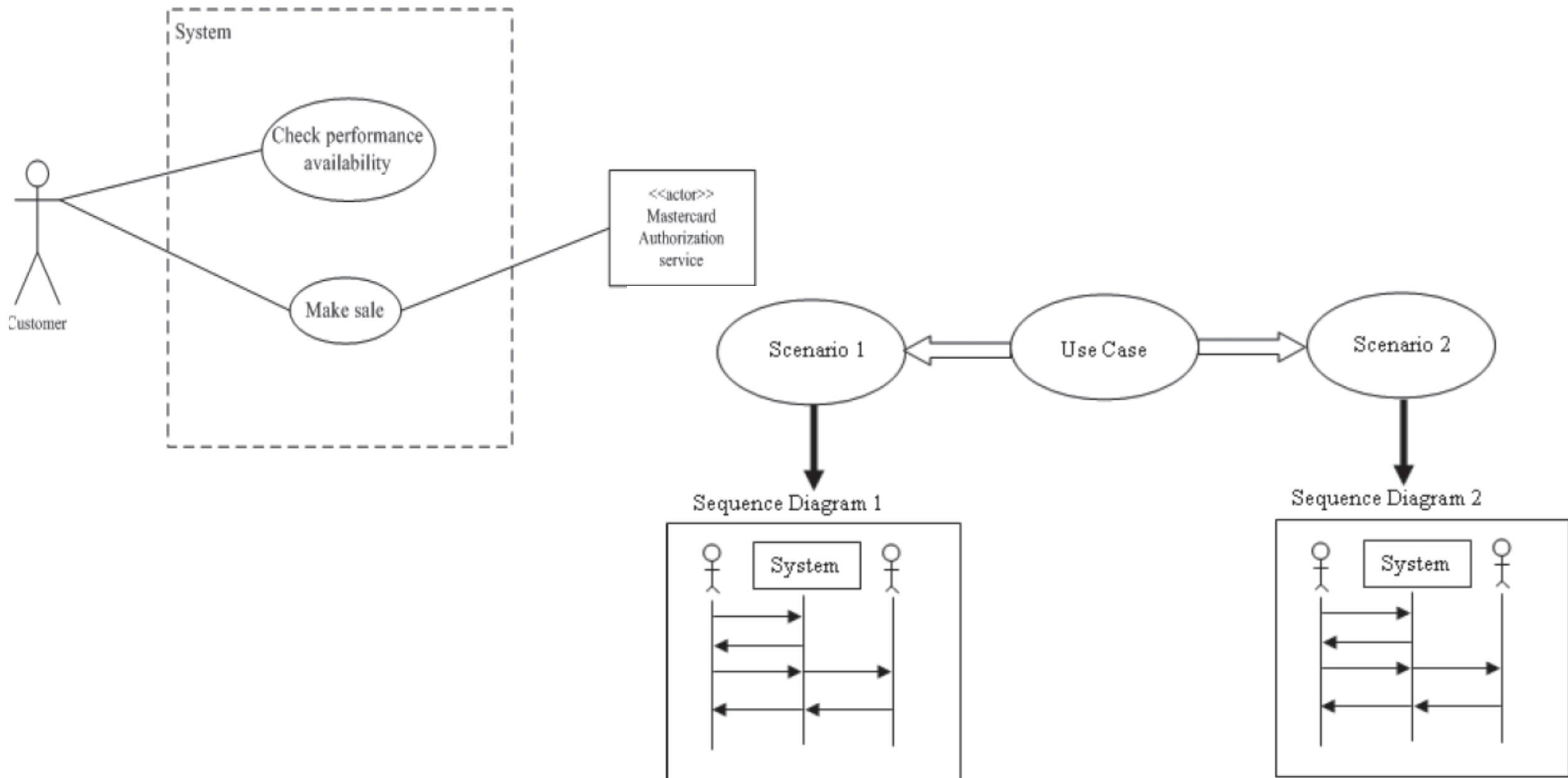


Figure 9.5 Transition from use cases to sequence diagrams.

2.Sequence Diagram

- One way to document a scenario is to write a text file description of what will happen under a given scenario. The text description should document all the steps that the actors will perform and the associated system responses.
- For example, consider a “normal” scenario in which a ticket sale is made. A possible description is as follows :

Scenario 1: Make Sale Use Case Normal Scenario

1. The customer requests a performance and seating preference from the ticket sale system.
2. The system verifies whether the performance and seating are available.
3. The performance and seating are available. The system offers the ticket to the customer.

2.Sequence Diagram

4. The customer accepts the ticket.
5. The system temporarily places the seating for that performance “on hold” while it processes the sale.
6. The system requests credit card information from the customer.
7. The customer supplies credit card information.
8. The system requests authorization to debit the credit card from the MasterCard authorization service (we will assume that MasterCard is the only acceptable form of payment).
9. The system receives the authorization approval.
10. The system confirms the sale to the customer and requests final acceptance of the transaction by the customer.

2.Sequence Diagram

11. The customer accepts the transaction.
12. The system bills the price of the ticket(s) to MasterCard.
13. MasterCard accepts the charge.
14. The system changes the status of the ticket from “on hold” to “sold.”
15. The system sends a final confirmation to the customer, which includes the sale transaction number that the customer will use to pick up the ticket at the box office prior to the performance (we don’t mail tickets).

2.Sequence Diagram

- Obviously, in this scenario, several twists and turns could have been introduced to upset the “normal” transaction sequence. The requested seating might not have been available (step 2), the credit card might have been invalid (step 9), or the customer might have changed his or her mind and canceled the transaction (step 11). These deviations would all have led to different scenarios.
- Once a typical scenario is documented, it is straightforward to convert it to a sequence diagram.
- A ***sequence diagram*** is a two-dimensional chart that displays the interaction between actors and the system across the horizontal direction and the sequencing, or timing, of that interaction in the vertical direction. The sequence diagram is basically a pictorial model of a scenario that was previously documented as text, as was done with scenario 1.

2. Sequence Diagram

- Next Figure is the sequence diagram of scenario 1. The objects are arranged across the top of the diagram. In this case, they include the customer, system, and MasterCard authorization service.
 - The vertical dashed lines beneath the objects are called *lifelines*. The lifeline shows the duration, or persistence, of the object during the scenario. The arcs with arrows indicate the direction of flow of the communication between objects. Some system arrows are self-loops because the system is communicating with itself. The precedence of the arrows from top to bottom shows the order (sequence) in which the communication transactions occur.
-  This is why it is called a sequence diagram

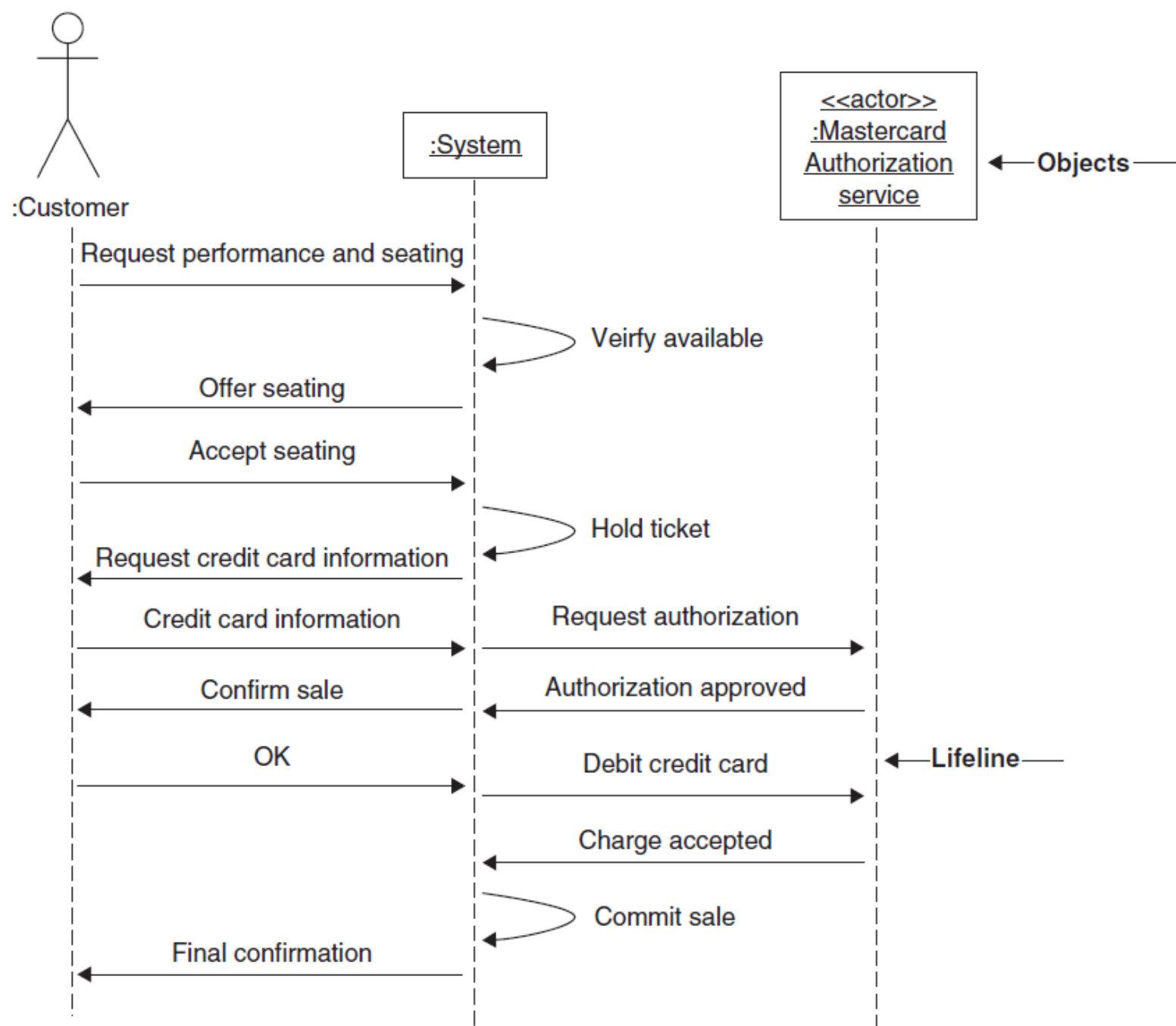


Figure 9.6 Sequence diagram of scenario 1.

2.Sequence Diagram

- Last Figure is a fairly high-level view of a sequence diagram. Embedded in the SYSTEM object are lower-level objects. Sequence diagrams can be opened up to uncover more objects and show more detail. In fact, this is part of the discovery process that occurs during the UML modeling phase, which intends to uncover object classes that can be coded independently of one another.
- Next Figure gives a more detailed view of the system by introducing the two main object classes of which the “**system**” is composed. When a user goes to a Web page for online sales, the customer enters a dialog with a system software component that resides on the Web page server. This software component deals with gathering information from the customer.

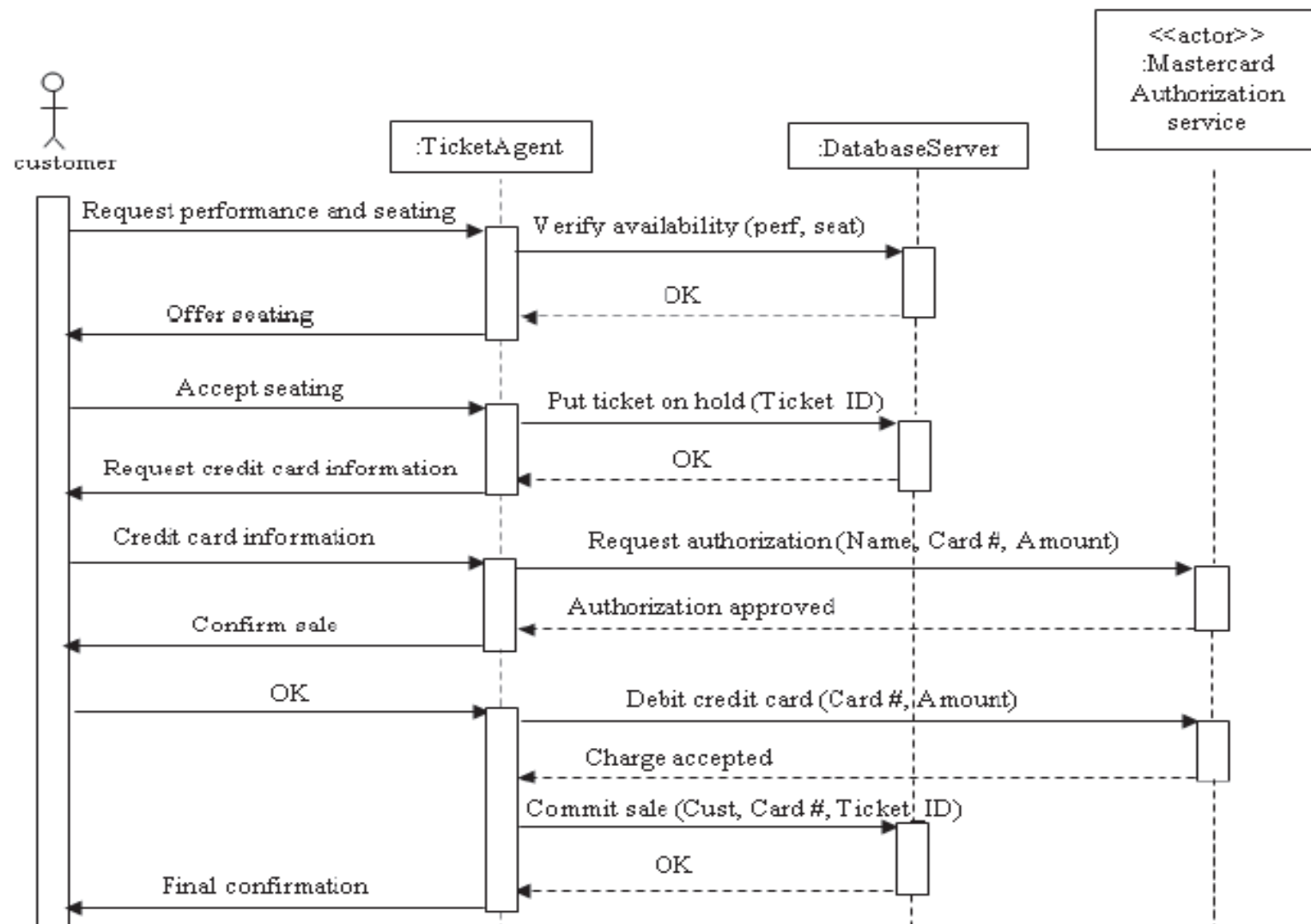


Figure 9.7 Sequence diagram with more details of scenario 1.

2.Sequence Diagram

- Let us refer to this software module as a **Ticket Agent object**. It is one component of what we have referred to as the **System object** in Figure 9.6. Information on performances, seating, ticket prices, and sales is kept in a database, which is another object class.
- The Ticket Agent object stores and retrieves information by sending SQL queries to the database server. Thus, in Figure 9.7, System object is broken down into two components: **Ticket Agent** and **Database Server**.
- The self-loops of Figure 9.6 are shown in Figure 9.7 as queries (functions) and responses between the ticket agent and the database server.
- Figure 9.7 is a more detailed view of scenario 1 as presented in the sequence diagram of Figure 9.6.

2.Sequence Diagram

Additional enhancements have been added in the last Figure 9.7:

The vertical rectangle boxes along the lifeline are **activation boxes**. They indicate the beginning and end of the activation of a procedure: The procedure usually begins with a message between objects, shown as an arrow having a solid line. The procedure ends with a return arrow to the initiator. Messages may be communications or they may be method calls.

If a method call returns a value, this is shown as an arrow with a dash line. Nested procedures are also possible, as shown in Figure 9.7.

For example, when Customer requests performance and seating information, Ticket Agent initiates a nested method call (verify availability ()). The nested call ends when a time value is returned (OK). Ticket Agent then offers the ticket to the customer. At that point this procedure ends.

2.Sequence Diagram

- If an analyst were to have a sequence diagram for each of the possible scenarios of the “make sale” use case, she or he would then know all the possible inputs and outputs of the ticket agent object, including all the database queries required in response to customer requests.
- From this information, the analyst could begin writing the software for the Ticket Agent class. Once the Ticket Agent class is designed and encoded as a module, it could be reused in any similar project for online ticket sales.
- Figure 9.7 could be further elaborated by introducing entity objects (tables) that will exist within the database. However, we shall stop at this level of abstraction for the time being.

3. Activity Diagram

- One of the behavioral design formalisms of UML is **the activity diagram**.
- An **activity diagram** is basically a flowchart that documents the flow of control between activities of the system. An **activity** is a state of the system in which some transaction or data computation takes place. It is represented by a box with rounded corners that has the name of the activity within it, as illustrated in Figure 9.8.
- Figure 9.8 is an activity diagram that represents the flow of control of scenario 1 and some other possible scenarios of the use case “make sale.” The activity diagram begins with a solid circle, the starting point. This leads to the first activity, “verify seating available,” and continues to other activities. The activity diagram has one or more terminal points, which are shown as circles with a solid dot.

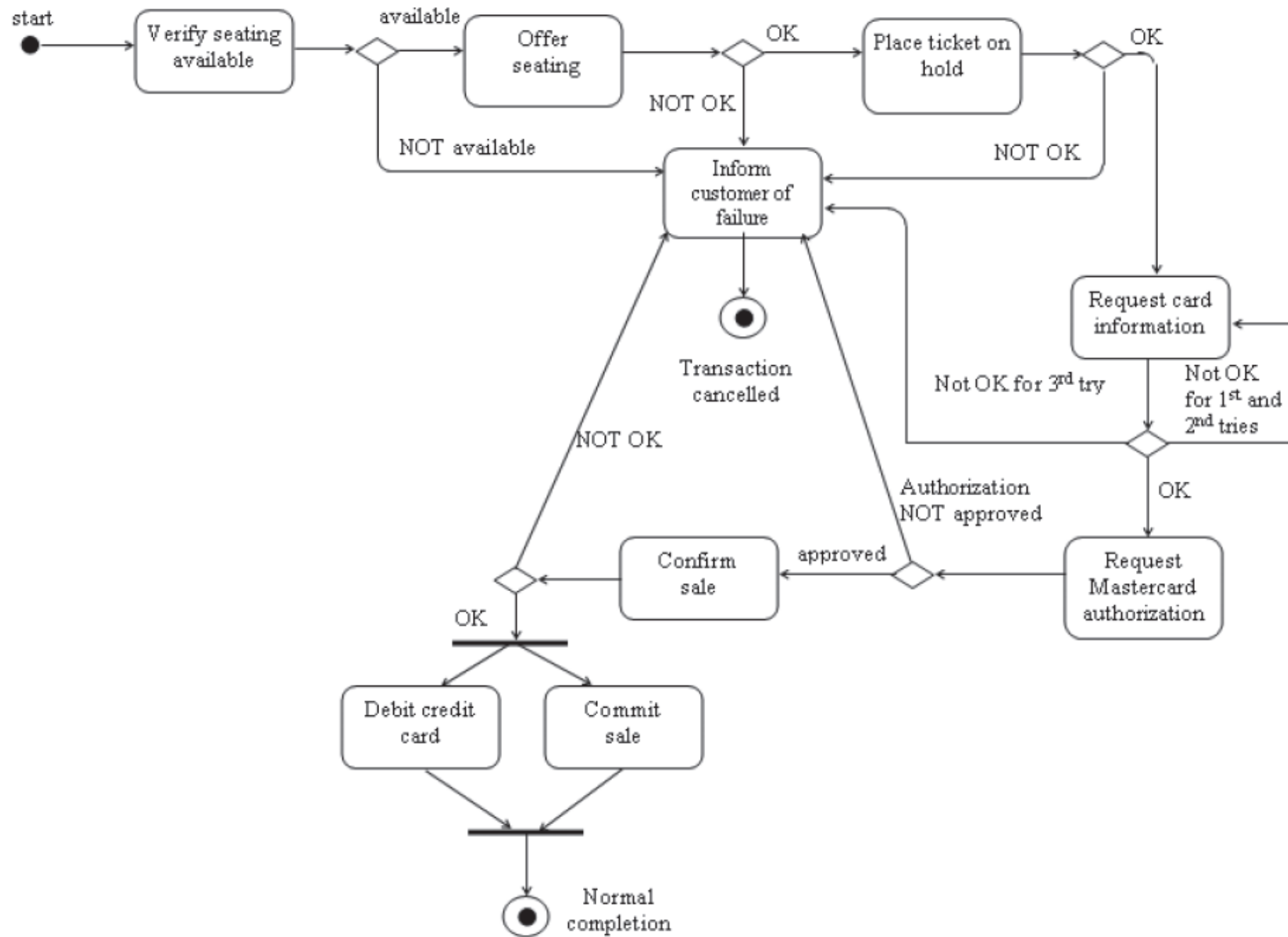


Figure 9.8 Activity diagram of some combined scenarios.

3.Activity Diagram

- In modeling the flow of control, activities are usually followed by decision points, shown as **diamonds**. A decision point models branching behavior when the flow of control is dependent on the outcome of the activity in the state prior to the decision point.
- ***For example:*** the first activity, “verify seating available,” can have two mutually exclusive results: the seating is available or it is not available. The labels on the output arcs of the decision point accommodate both cases in the flow of control. If the seating is available, the next activity performed by the system is “offer seating.” However, if the seating is not available, the next activity is “inform the customer of failure,” followed by a termination, or canceling of the transaction. Presumably, the customer can try again with another seating request.

3.Activity Diagram

The directed arcs show the direction of the flow of control. All arcs in an activity diagram are directed arcs.

From Figure 9.8, one can trace the execution of scenario 1, the normal scenario for making a reservation. The string of activities of scenario 1 is as follows:

start → verify seating available → offer seating → place ticket on hold → request card information → request MasterCard authorization → confirm sale → commit sale and debit credit card → normal completion.

3. Activity Diagram

Note the following details about this string of activities:

- 1.** It begins with the first activity *within the system*. In the sequence diagram shown in Figure 9.7, the first event within the system is labeled “Verify availability.”
- 2.** It includes all transactions and operations of scenario 1 *carried out by the system*.
- 3.** Each activity incorporates one or more sequence transactions, (e.g., “Request MasterCard authorization” incorporates the sequence “Request authorization” and “Authorization approved” of Figure 9.7).

3. Activity Diagram

- Activity diagrams look somewhat like flowcharts in procedural programming. However, unlike flowcharts, activity diagrams can model concurrency (i.e., the execution of more than one activity simultaneously).
- Consider the last two activities of Figure 9.8. When a sale is confirmed, the activity diagram enters a fork, shown as a bar.
- A **fork** has one input arc and more than one output arc.
- This leads to the activation of more than one activity in parallel. When the simultaneous and independent activities are complete, they enter a join.
- A **join** has two or more input arcs and one output arc. The activity that follows a join can begin when all input activities are complete.

3. Activity Diagram

- The activity diagram of Figure 9.8 includes scenario 1 and other possible scenarios.

Consider, for example, the following string of activities: start → verify seating available → inform customer of failure → transaction canceled. This, too, is a possible scenario of the use case “make sale.”

- This scenario can be modeled in a sequence diagram such as Figure 9.9. In fact, when an activity diagram is used to document a use case, the activity diagram should cover all of the scenarios that might occur in the execution of a use case.

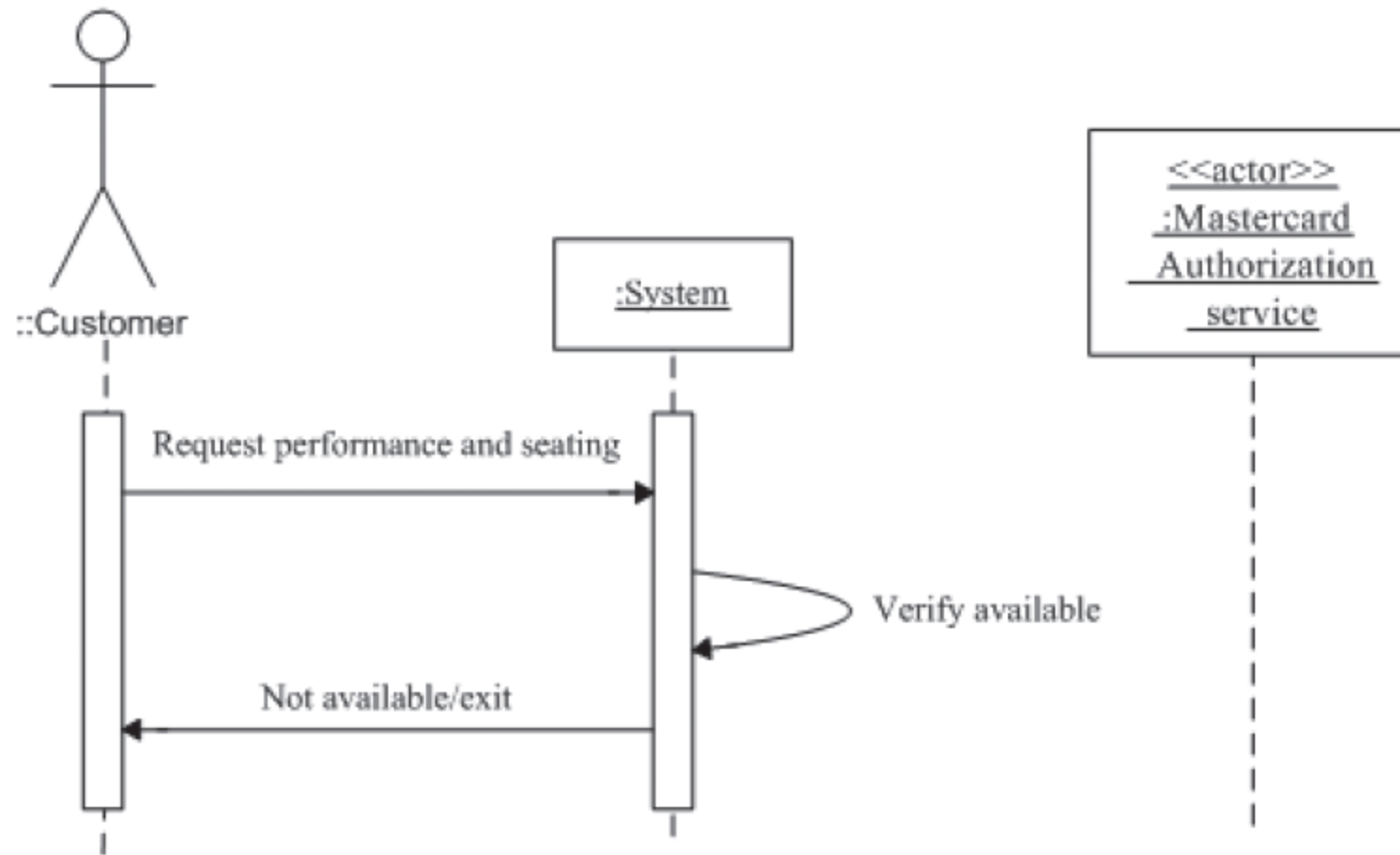


Figure 9.9 Sequence diagram for scenario 2.

4. Case study

The scenario is to “Confirm Validity of Shipment”. When a delivery truck arrives at the receiving dock, the receiving clerk is responsible for clearing the material for unloading. The clerk first compares the paperwork that comes with the shipment with the enterprise’s purchase order that is referenced on the bill of lading. Therefore, two required information inputs to this function are the bill of lading and the corresponding purchase order. If the material on the shipment bill of lading corresponds to the purchase order, the shipment is accepted. If no such purchase order exists, the shipment is refused. This procedure is defined as the “material receiving procedure.” The steps of this procedure, as defined by the management of the enterprise, control the manner in which the function is performed.

4. Case study

Two actors interface with the system (database) when using the purchase order in various use cases:

The purchasing agent **creates** the purchase order. This includes all of the header information in the Purchase Order table and the detail information in the PO_Detail table.

After being created, the purchase order **is printed** and sent to the vendor.

No further use is made of the purchase order information until material is delivered. At that time, the receiving clerk compares the shipment information to the purchase order (**Confirm** Validity of Shipment) and prepares a receiving report or otherwise informs the purchasing agent of the delivery against the purchase order (**Notify** Purchasing Agent).

The purchasing agent will then change the purchase order balances by subtracting the amount received by line item from the current balance (**Update** the PO balances).

Finally, when all line items have been delivered, the purchasing agent will **close** the purchase order.

Use Case Diagram

4. Case study

Establish **the sequence diagram** for the normal success scenario of the use case “**Confirm Validity of Shipment.**” The receiving clerk must first login to the DBMS.

Then the purchase order number is submitted via a user form, and a confirmation of the PO number and vendor ID are returned to be compared with the bill of lading.

The line items are also retrieved for comparison with the shipment details.

Depending on how the forms are implemented, two queries can be used or a single query can return a master/detail record for the entire purchase order.

Regardless of implementation details, the order of reviewing content is as suggested in the sequence diagram. Finally, the receiving clerk closes the form.

4.Case study

4.Case study

The activity diagram represents the various scenarios of the use case from the system perspective.

Unlike the Ticket Sale example, there are no agents or other software objects in the “Confirm Validity of Shipment” use case.

Decisions on whether or not to accept materials are made by the receiving clerk, who is external to the system.

After logging in, the receiving clerk enters the PO number from the bill of lading. The system (database) either returns the purchase order from the database or indicates that no record with that PO number is found. A false result terminates the activity. A true result is followed by an examination of the PO detail.

4. Case study

Since a purchase order will always have a detail, the record will always be returned. From the system point of view, this is followed by a termination of activity.

Now consider the possibility that the PO detail may not correspond to items on the bill of lading. The receiving clerk has a decision to make (i.e., whether to accept or not accept the shipment). But this is a decision that is *made outside the system*. So no decision node is included for that decision.

If the designers had decided to include a software object within the system that would take the information on the bill of lading and make the decision as to whether or not to accept the shipment. So the activity diagram reflects only decisions made by the system.

4.Case study

Use Case: "Make Sale"



This use case can have MULTIPLE scenarios:

- Scenario 1: Normal scenario (everything goes smoothly)
 - Scenario 2: Credit card rejected
 - Scenario 3: Seating not available



Each scenario gets written out in detail:

- Scenario 1 detailed steps:
 1. Customer requests performance and seating
 2. System verifies availability
 3. System offers seating ... (all 15 steps like in slides)



Then we convert this detailed scenario into a Sequence Diagram



All Scenarios Combined into One Activity Diagram

One Last Thought

- **Use Case Diagram:** Shows WHAT functions the system provides (the use cases) and WHO uses them (the actors).
 - Example: "Make Sale" use case
- **Sequence Diagram:** Shows HOW objects interact to execute ONE specific scenario of that use case, with detailed message-by-message interactions.
 - Example: Sequence diagram for "Make Sale - Normal Scenario" (where everything goes right)
 - Example: Sequence diagram for "Make Sale - Credit Card Rejected Scenario"
 - Each sequence diagram = ONE path through the use case
- **Activity Diagram:** Shows the WORKFLOW and LOGIC covering ALL possible scenarios of that use case, including all decision points and branches.
 - Example: Activity diagram for "Make Sale" showing ALL paths:
 - The normal path (success)
 - The "seating not available" path
 - The "credit card rejected" path
 - The "customer cancels" path
 - All other possible branches